MinorMap+ PROJECT SCOPE DOCUMENT

Smart Minor Eligibility & Recommendation Engine

Team Name: 500 OK
Team Members:

- **Ibrahim** – Backend Developer
- **Anil** – Authentication Specialist
- **Janci** – Database Manager
- **Rashika** – Team Lead, UI Developer & Tester

Date: November 11, 2025
Course: Backend Development Project Day

Section 1: Project Overview, Objectives, and Target Users

PROJECT OVERVIEW

MinorMap+ is a **smart backend system** that evaluates a student's eligibility for different minors and career tracks using an intelligent quiz-based recommendation engine.
Students log in, select a minor, take an eligibility test, and instantly receive results showing whether they are Eligible, Potential, or Not Recommended.

The system uses a **trait-weighted algorithm** to match answers to nine available minors:
AI & ML, **Data Science**, **IoT**, **Product Management & Design**, **FinTech & Blockchain**, **Blockchain & Cybersecurity**, **Life Sciences**, **Energy Sciences**, and **Entrepreneurship**.

Built with **Node.js**, **Express.js**, **MongoDB**, and **JWT**, the platform focuses on secure authentication, reliable backend logic, and explainable results.

PROJECT OBJECTIVES

1. Develop a secure backend system for user login, quiz submission, and eligibility computation.
2. Implement a weighted scoring algorithm to evaluate student suitability across multiple minors.
3. Allow admin users to manage questions, weight mappings, and minor data.
4. Design RESTful APIs for authentication, quiz handling, and result reporting.
5. Create minimal HTML/CSS interfaces for login, minor selection, quiz, and results.
6. Test all endpoints thoroughly using Postman with proper authentication.
7. Implement validation, logging, and error-handling middleware for data accuracy and security.

TARGET USERS

1. **University Students** – Discover which minor best fits their strengths and interests.
2. **Faculty Advisors** – Analyze student eligibility and interest trends.
3. **Career Counselors** – Use data-driven recommendations to guide students.
4. **Developers/Learners** – Explore scoring algorithms and secure backend design.

Section 2: Technology Stack

TECHNOLOGY STACK

Backend Framework:

- **Node.js:** JavaScript runtime environment for backend logic.
- **Express.js:** RESTful API framework.
- **Mongoose:** ODM library for MongoDB integration.

Database:

- **MongoDB:** NoSQL database for storing user, quiz, and recommendation data.
- **Database Name:** `MinorMap_DB`
- **Collections:** users, questions, responses, recommendations, minors.

Authentication & Security:

- **JWT (JSON Web Token):** Secure authentication and route protection.
- **bcryptjs:** Password hashing for user accounts.
- **dotenv:** For environment configuration.
- **CORS:** To allow frontend–backend communication.

Frontend:

- **HTML5 / CSS3:** Basic pages (Register, Login, Select Minor, Quiz, Results).
- **JavaScript:** API calls and token-based communication.

Development Tools:

- **Postman:** Endpoint testing and documentation.
- **VS Code:** Code editor.
- **Git / GitHub:** Version control.
- **npm:** Package management.

Section 3: Functional Modules and API Endpoints

FUNCTIONAL MODULES

1. Authentication Module

Register User

- **POST /api/auth/register**
- Create a new student account.
- **Body:** `{ name, email, password }`

Login User

- **POST /api/auth/login**
- Authenticate and return JWT.
- **Body:** `{ email, password }`

---

2. Minor & Question Management (Admin Only)

Get All Minors

- **GET /api/minors**
- Fetch list of available minors.

Add New Question

- **POST /api/admin/questions**
- Add quiz question and weight mapping per minor.

Edit Question / Minor

- **PUT /api/admin/questions/:id** or **PUT /api/admin/minors/:id**
- Modify content or weights.

---

3. Eligibility Test Module

Fetch Questions

- **GET /api/questions**
- Retrieve active question set for the test.

Submit Answers

- **POST /api/test/submit**
- Submit answers, backend computes eligibility.
- **Body:** `{ answers: [{ questionId, value }] }`

---

4. Recommendation Module

Get User Recommendations

- **GET /api/recommendations/:userId**
- Fetch previous eligibility test results.

Download Report

- **GET /api/report/:recId**
- Generate a PDF or JSON report of the test results.

---

5. Analytics Module (Admin)

Get Trends

- **GET /api/admin/analytics**
- Display the popularity of minors and average scores.

SCORING LOGIC

- Each question has weighted mappings to multiple minors.
- For each minor:
  `score = (sum(answerValue × weight) / maxPossibleWeight) × 10`
- Thresholds:
  - **≥8.0** – Eligible
  - **6.0–7.9** – Potential
  - **<6.0** – Not Recommended
- Top 3 minors displayed; highest = “Best Fit”.

Section 4: Roles and Responsibilities

TEAM STRUCTURE

Team Name: 500 OK
Team Size: 4 Members
Project Name: MinorMap+

Member	Role	Responsibilities	Contribution
Ibrahim	Team Lead & Backend Developer	API development, controller logic, scoring algorithm, DB integration	30%
Anil	Authentication Specialist	JWT implementation, token management, security middleware, password hashing	25%
Janci	Database Manager	MongoDB schema design, data validation, question mappings, Postman testing	25%
Rashika	UI Developer	HTML/CSS forms, frontend-backend integration, API testing, documentation	20%

COLLABORATIVE RESPONSIBILITIES

- Daily standups and progress updates.
- Code documentation and review.
- Bug fixing, testing, and debugging.
- Final packaging and submission.

Section 5: Project Timeline and Expected Deliverables

EXPECTED DELIVERABLES

1. **Project Scope Document (PDF)**
    - 3-page overview including objectives, stack, modules, and roles.

2. **Working Backend API**

```
Team_500OK_MinorMap+_Project/
├── backend/
│   ├── models/ (User.js, Question.js, Response.js)
│   ├── controllers/ (authController.js, testController.js)
│   ├── routes/ (authRoutes.js, testRoutes.js)
│   ├── middleware/ (authVerify, logger, errorHandler)
│   ├── config/ (db.js)
│   └── server.js
```

```
├── frontend/
│   ├── Register.html
│   ├── Login.html
│   ├── SelectMinor.html
│   ├── Quiz.html
│   └── Results.html
├── postman_collection.json
├── Project_Scope.pdf
├── Submission_Report.pdf
└── README.md
```

2.
3. **HTML/CSS Frontend (5 Pages)**
    - Register, Login, Select Minor, Quiz, Results
4. **Postman Collection (.json)**
    - Auth + Quiz + Recommendation routes
      Token tests and validation
5. **Submission Report (PDF)**
    - Roles, reflection answers, screenshots, and challenges faced

---

DAILY MILESTONES

Day	Key Milestone	Completion %
Day 1	Planning & Auth Module	20%
Day 2	Database Setup & Scoring Engine	50%
Day 3	Frontend & API Integration	75%
Day 4	Testing & Error Handling	90%
Day 5	Documentation & Submission	100%

CRITICAL DEADLINES

- **Day 1, 6:00 PM** — Scope Document Submission
- **Day 3, 6:00 PM** — Backend + Frontend Functional
- **Day 5, 6:00 PM** — Final Project Submission

QUALITY STANDARDS

- Clean, well-commented MVC architecture
- All routes tested in Postman
- JWT authentication & secure passwords
- Proper error handling and validation
- Responsive HTML pages
- Complete documentation

TEAM COLLABORATION

- Daily 15-min standups
- Code reviews before merge
- End-of-day progress reporting
- Pair programming for scoring logic

Total Project Duration: 5 Days
Estimated Effort: 40 hours per team member
Final Submission Deadline: Day 5, 6:00 PM